SRI LANKA INSTITUTE OF INFORMATION TECHNOLOGY

Faculty of Computing

IE3112 – MOBILE SECURITY

REVERSE ENGINEERING AND CONTROLLED MALWARE INJECTION IN ANDROID APPLICATIONS (EDUCATIONAL TROJAN SIMULATION)

Group Number: 09
Member Details:

Bachelor of Science in Information Technology

Department of Information Technology

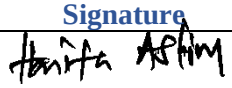
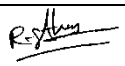
Sri Lanka Institute of Information Technology

Sri Lanka

2026

DECLARATION

We declare that this is our own work and this report does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
MHM.Aslim		
Sathursha.R		
Rachana.M		
Raksha.M		

The above candidates are carrying out research for the undergraduate assignment under my supervision.

Supervisor: Mr. Tharaniya Warma

Signature of the Supervisor: _____

Date: _____

ABSTRACT

This report documents the reverse engineering and controlled malware injection performed on the Privacy Friendly flashlight Android application (com.secuso.torchlight2) as part of the IE3112 Mobile Security assignment. The original APK was decompiled using Apktool and its structure, components, permissions, and logic flow were systematically analysed. Two Trojan-like components were subsequently injected into the decompiled Smali bytecode without disrupting the original application functionality.

The first component silently logs device information including the device manufacturer, model, brand, and Android version to a file on device storage upon every application launch. The second component registers a BroadcastReceiver programmatically within the MainActivity to monitor network connectivity changes and fires contextual notifications indicating whether the device connected to WiFi, connected to mobile data, or lost network connectivity entirely.

The report details the step by step injection process, permission modifications made to the AndroidManifest.xml, trigger logic implementation, recompilation and signing procedures, and the challenges overcome during development including the Android 8 restriction on manifest-registered CONNECTIVITY_CHANGE receivers. A security analysis section evaluates how such attacks could be detected and prevented, identifies defensive strategies for developers, and proposes mitigation strategies for both end users and organisations.

Keywords: Android security, reverse engineering, Smali bytecode, Trojan simulation, BroadcastReceiver, APK modification, mobile malware analysis

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: REVERSE ENGINEERING

1.1 APK Structure Analysis

The Privacy Friendly Flashlight application (com.secuso.torchlight2) was obtained as a free and open-source APK from the F-Droid repository. The application was selected because it is a simple, single-purpose flashlight utility with no sensitive user data, no network communication in its original form, and no authentication mechanisms, making it an ideal candidate for a controlled academic exercise.

The APK was decompiled using Apktool version 2.9 with the following command:

```
./apktool d torchlight.apk -o torchlight_decoded
```

The decompiled output produced the following top-level directory structure:

- smali/ - Dalvik bytecode for all Java/Kotlin classes translated into human-readable Smali assembly format
- res/ - All application resources including layouts, drawables, strings, colours, and values
- AndroidManifest.xml - Application component declarations, permissions, and metadata
- apktool.yml - Apktool metadata including original SDK versions and resource identifiers

The original APK was compiled against Android SDK version 34 (Android 14) and declared a minimum SDK version of 21 (Android 5.0), giving it a wide device compatibility range. The main application package was com.secuso.torchlight2, with an additional dependency package org.secuso.pfacore providing the Privacy Friendly App Core library used for settings, about, help, and tutorial screens. The Smali source was partially processed by R8 minification, which resulted in some internal utility classes having non-descriptive names, however the primary activity classes retained meaningful identifiers.

1.2 Component Mapping

Analysis of the AndroidManifest.xml revealed the following application components registered in the original application prior to any modification.

Component Type	Class Name	Exported	Purpose
Activity	SplashActivity	true	Launcher entry point
Activity	MainActivity	false	Flashlight control
Activity	TutorialActivity	false	First-run tutorial (pfacore)
Activity	AboutActivity	false	About screen (pfacore)
Activity	HelpActivity	false	Help screen (pfacore)
Activity	SettingsActivity	false	Settings (pfacore)
Activity	ErrorReportActivity	false	Error reporting (pfacore)
Service	SystemAlarmService	false	WorkManager alarm
Service	SystemJobService	true	WorkManager job
Service	SystemForegroundService	false	WorkManager foreground
Receiver	ConstraintProxy$NetworkStateProxy	false	WorkManager (disabled)
Receiver	DiagnosticsReceiver	true	WorkManager diagnostics
Receiver	ProfileInstallReceiver	true	AndroidX profile installer

Table 1.1: Application Components in Original APK

The application defined seven Activity components. SplashActivity served as the launcher entry point declared with android:exported="true" and the MAIN/LAUNCHER intent filters. MainActivity contained all primary flashlight control logic. The remaining five activities were contributed by the pfacore library. Three background Service components belonged entirely to the AndroidX

WorkManager library. Nine BroadcastReceiver components were registered, all originating from WorkManager and ProfileInstaller libraries. No custom services or custom receivers were present in the original application.

1.3 Control Flow Explanation

The application launch flow begins at SplashActivity, which performs pfacore library initialisation and transitions to MainActivity. The MainActivity class extends BaseActivity, which extends the pfacore DrawerActivity, providing the navigation drawer and settings integration.

Within MainActivity, the onCreate() method is the primary initialisation point. It calls setContent() to inflate the main layout, initialises the flashState boolean field to false, creates a PFAPermissionAcquirer object to handle camera permission requests at runtime, locates the flashlight toggle CheckBox from the layout, sets its initial state from the closeOnPause user preference, and attaches a click listener. Finally, onCreate() calls init() to establish the camera connection.

The init() method establishes a connection to the device camera hardware through the ICamera interface abstraction, which supports both Camera1 and Camera2 APIs. When the user toggles the flashlight button, the toggleCamera lambda is invoked, which calls ICamera.toggle() to switch the flash state and updates the flashState field accordingly.

The onResume() lifecycle method checks the isConnected field and calls init() again if the camera connection was lost, ensuring reconnection after the activity returns from background. The onStop() method releases camera hardware and resets connection state, optionally toggling the flash off based on the closeOnPause preference. The onKeyDown() method intercepts the Back button and calls moveTaskToBack() rather than finishing the activity, keeping it in the background task stack.

1.4 Permission Analysis

Permission	Type	Purpose in Original App
android.permission.CAMERA	Dangerous	Access device camera hardware for flashlight control
android.hardware.camera	Feature	Hardware feature declaration for Play Store filtering
android.permission.FLASHLIGHT	Normal	Direct flashlight hardware access on supported devices

Table 1.2: Permission Analysis of Original APK

The original application requested only three permissions, all directly related to its core flashlight functionality. No network, storage, location, or notification permissions were present in the original manifest. This minimal permission footprint made the application a privacy-respecting utility consistent with its classification under the Privacy Friendly Apps project by the SECUSO research group.

1.5 Identified Vulnerabilities

Several weaknesses were identified during the reverse engineering phase that made the application susceptible to the injection attack performed in Phase 2.

1.5.1 Insufficient Code Obfuscation

The application had minimal obfuscation beyond basic R8 processing. Primary classes including MainActivity, BaseActivity, and PFTorchlight retained meaningful class and method names in the decompiled Smali output. An attacker familiar with Android Smali bytecode could readily navigate the decompiled code and identify injection points such as the onCreate() method without significant effort.

1.5.2 Absence of Runtime Integrity Verification

The application performed no APK signature or integrity verification at runtime. Once the APK was decompiled, modified, and re-signed with a new keystore generated by the group, the modified version installed and executed without any resistance from the application. There was no mechanism to detect that the signing certificate had changed from the original developer's certificate.

1.5.3 No Anti-Debugging or Tamper Detection

The original manifest did not include runtime tamper detection. The absence of checks such as android.os.Debug.isDebuggerConnected() or signature verification allowed the injected code to execute without any detection mechanism in place.

1.5.4 Over-broad Exported Components from Dependencies

The application's inclusion of the AndroidX WorkManager and ProfileInstaller libraries introduced several exported receivers protected only by the android.permission.DUMP permission. While this is standard library behaviour, it expands the attack surface of the application beyond what its core flashlight functionality strictly requires.

CHAPTER 2: MALWARE INJECTION IMPLEMENTATION

2.1 Overview of Injected Components

Phase 2 involved modifying the decompiled Privacy Friendly Torchlight application to simulate Trojan-like behaviour without altering any visible functionality. The user continues to see a normal working flashlight application. Two malicious components were injected entirely at the Smali bytecode level with no access to the original Java or Kotlin source code.

The first injected component is a device information logger embedded directly inside the existing MainActivity.onCreate() method. It silently collects device metadata and writes it to a file on device storage upon every application launch. The second injected component is a new BroadcastReceiver class named NetworkChangeReceiver that monitors network connectivity changes and delivers contextual notifications to the device status bar, correctly identifying whether the device has connected to WiFi, connected to mobile data, or lost all network connectivity.

2.2 Step by Step Injection Process

2.2.1 Step 1 - Decompiling the APK

The original APK was decompiled using Apktool with the following command executed from the Windows command prompt on the development machine:

```
./apktool d torchlight.apk -o torchlight_decoded
```

The decompilation produced the full directory structure shown in Figure 2.1 below, including all Smali source files organised by package, the AndroidManifest.xml, and all resource files in their original form.

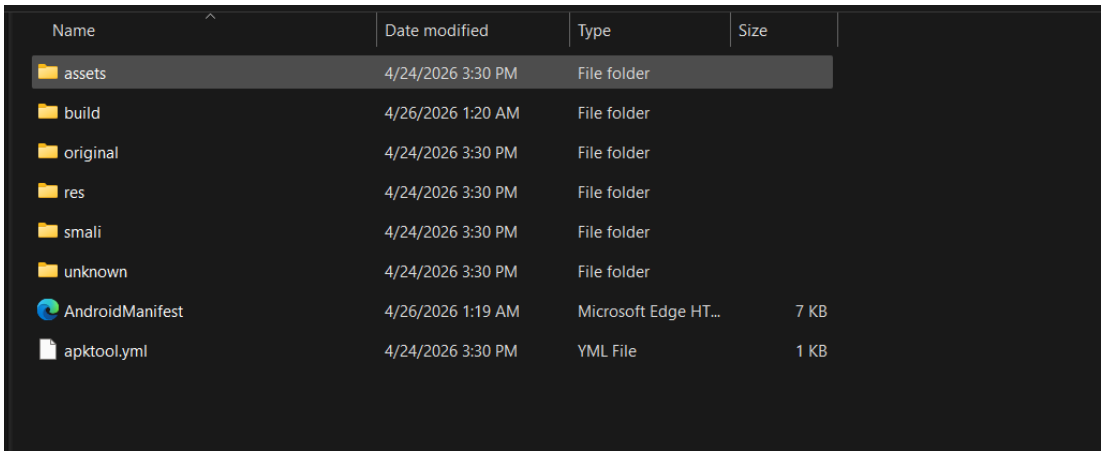


Figure 2.1: APK decompiled directory structure

2.2.2 Step 2 - Analysing the Original MainActivity

The original MainActivity.smali was opened and examined to identify the onCreate() method entry point. In the original unmodified file, the onCreate() method performed the following operations in sequence: called the parent class onCreate() via invoke-super, called setContent() to inflate the flashlight layout, initialised the flashState field to false, created a PFAPermissionAcquirer for camera permission handling, set up the CheckBox toggle, and finally called init() followed immediately by return-void. There was no data collection, no file writing, and no receiver registration of any kind in the original code, confirming that the method was a clean and safe injection target.

2.2.3. Group Identification Toast Popup

As the first action within the injected onCreate() block, a Toast notification was added to display group identification details on every application launch. This satisfies the demonstration requirement and confirms that the modified APK is running on the target device.

The injected Smali code for the Toast is:

```
# Popup for Group 09 Assignment
const-string v0, "Group 09 - IT23642300, IT23584204, IT23833920, IT23834088"
const/4 v1, 0x1
invoke-static {p0, v0, v1}, Landroid/widget/Toast;->makeText(Landroid/content/Context;Ljava/lang/CharSequence;I)Landroid/widget/Toast;
move-result-object v0
invoke-virtual {v0}, Landroid/widget/Toast;->show()V
```

Figure 2.2: Injected Smali code for Toast notification

2.2.4 Step 3 - Injecting the Device Information Logger

The first Trojan component was injected directly into the onCreate() method of MainActivity.smali, inserted immediately before the original return-void instruction. The injected block is wrapped in a try-catch structure using .catch Ljava/lang/Exception to ensure the original application continues functioning even if the logging fails.

Inside the try block, the following operations are performed silently on every application launch. A SimpleDateFormat object is instantiated with the pattern yyyy-MM-dd HH:mm:ss and used to format the current system time from a new Date() object. A StringBuilder then assembles a multi-line log string containing the device log header, the formatted timestamp under the label Log Time, the device manufacturer from android.os.Build.MANUFACTURER, the device model from android.os.Build.MODEL, the device brand from android.os.Build.BRAND, and the Android version from android.os.Build.VERSION.RELEASE. The completed string is written to the path /sdcard/Download/group09_log.txt using a FileOutputStream, and the stream is then closed. The catch block at label :catch_0 silently absorbs any IOException and falls through to return-void.

The complete injected Smali code for the device logger is as follows:

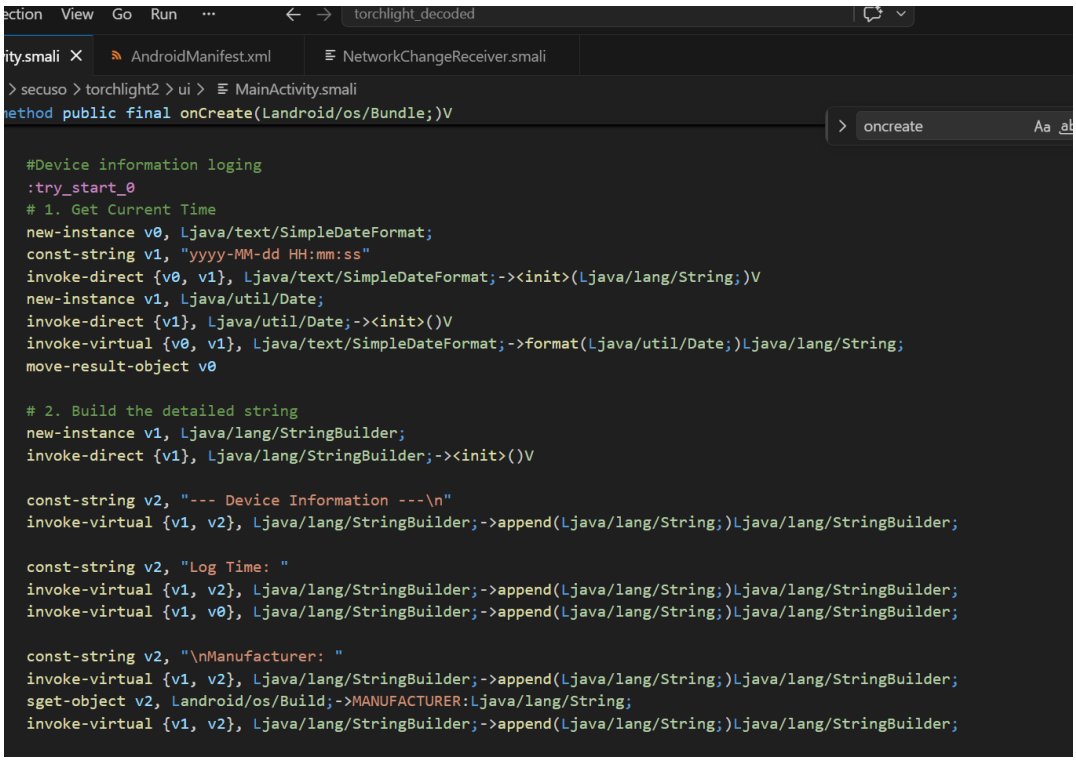


Figure 2.3: Injected Smali code for Device information logger(part1)

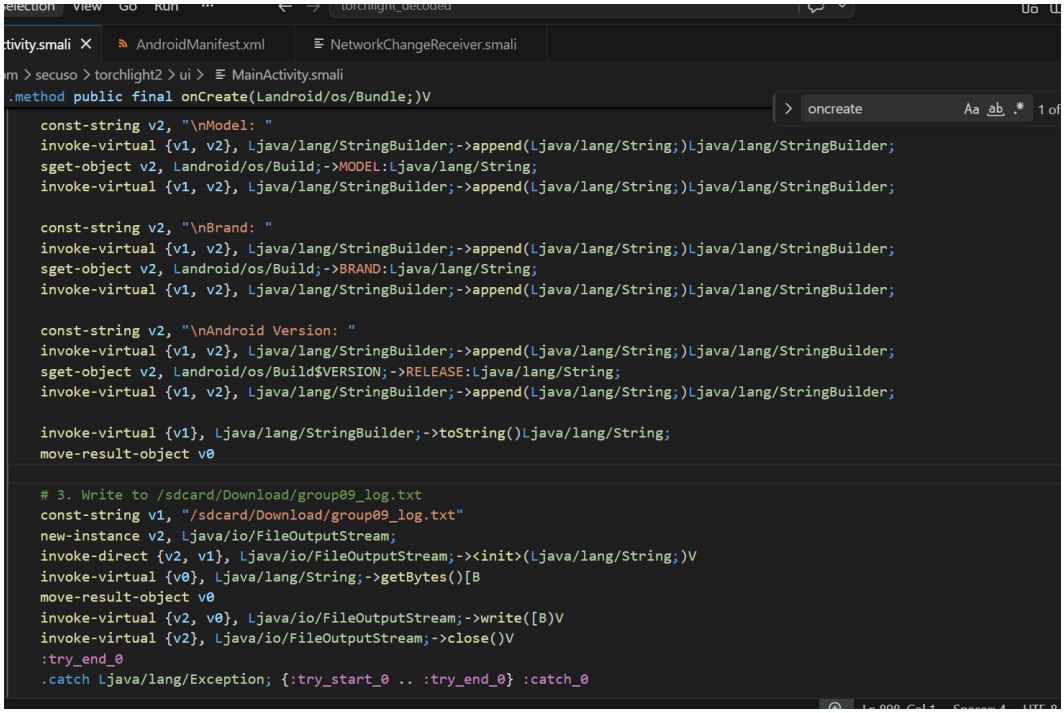


Figure 2.4: Injected Smali code for Device information logger(part2)

2.2.5 Step 4 — Creating NetworkChangeReceiver.smali

The second injected component required creating an entirely new Smali file at the path:

smali/com/secuso/torchlight2/ui/NetworkChangeReceiver.smali

This file declares a new class that extends android.content.BroadcastReceiver. The class header and constructor are defined as follows:

```
.class public Lcom/secuso/torchlight2/ui/NetworkChangeReceiver;
.super Landroid/content/BroadcastReceiver;

.method public constructor <init>()V
    .locals 0
    invoke-direct {p0}, Landroid/content/BroadcastReceiver;
    -><init>()V
    return-void
.end method

.method public onReceive(Landroid/content/Context;Landroid/
content/Intent;)V
    .locals 8
```

Figure 2.5: NetworkChangerReceiver.smali – class header and field declaration

The onReceive() method first obtains the NotificationManager system service using getSystemService("notification"). It then creates a notification channel with the channel ID group09_channel, the channel name Security Alerts, and importance level IMPORTANCE_HIGH (value 4) inside a try-catch block to handle devices running Android versions below 8.0 gracefully. After channel creation, the method obtains the ConnectivityManager using getSystemService("connectivity") and calls getActiveNetworkInfo() to retrieve the current network state object.

The network type decision logic is as follows. The default title and body text variables are pre-set to No Network Connection and the corresponding description. If the NetworkInfo object returned by getActiveNetworkInfo() is null, execution jumps directly to the :send_notification label with the disconnected message. If the object is non-null and isConnected() returns true, the method calls getType() which returns an

integer: 0 for WiFi and 1 for mobile data. If the type equals 1, execution branches to :is_mobile and sets the title to Mobile Data Connected. Otherwise WiFi Connected is used as the fall-through case. The notification is then built using the native android.app.Notification$Builder with the system icon resource 0x01080020, the dynamic title string, and the dynamic body string, and delivered via NotificationManager.notify().

The network type check section of the code is as follows:

Figure 2.6: NetworkChangerReceiver.smali – Network type decision logic (part1)

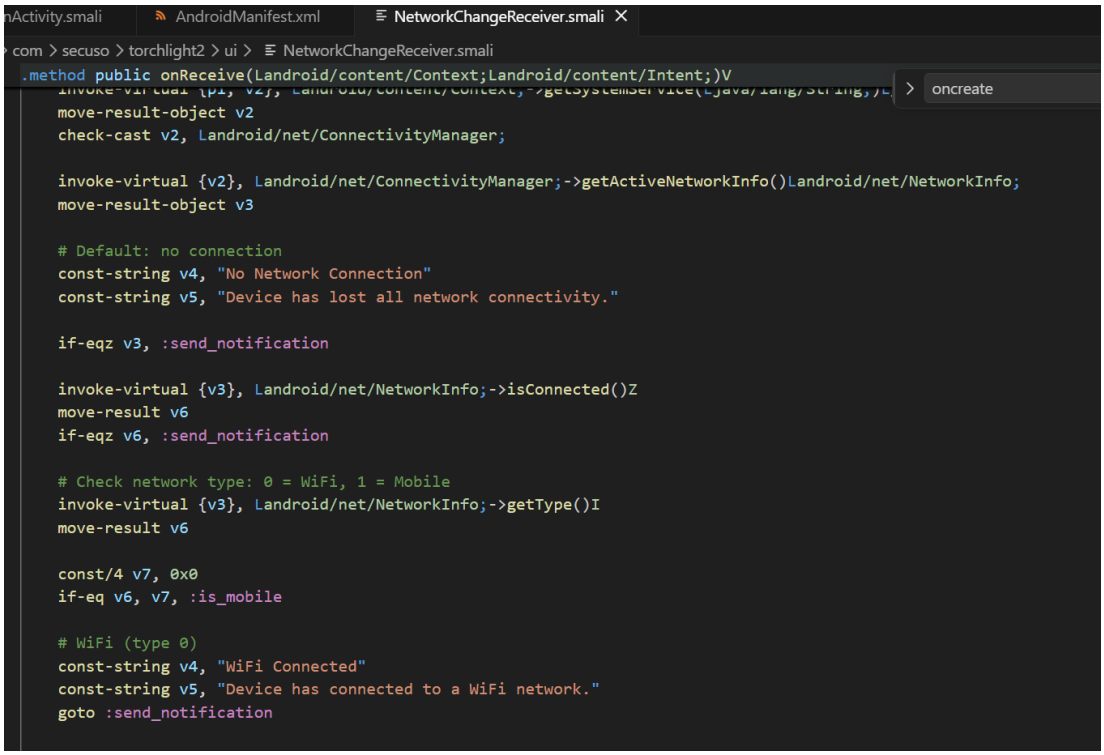


Figure 2.7: NetworkChangerReceiver.smali – Network type decision logic (part2)

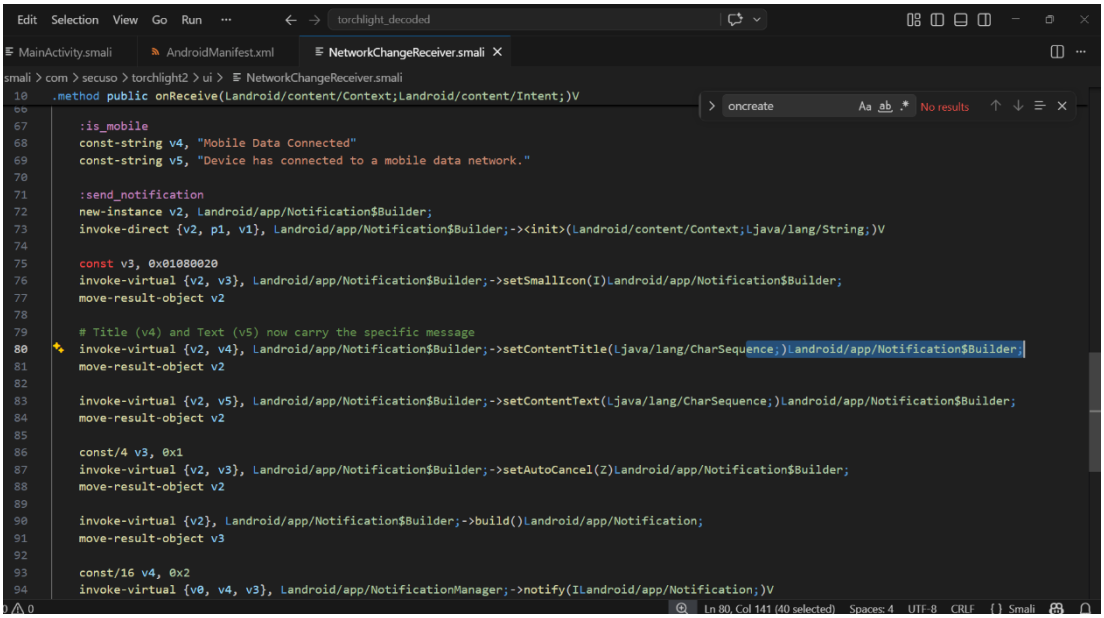


Figure 2.8: NetworkChangerReceiver.smali – Network type decision logic (part3)

```
    const/16 v4, 0x2
    invoke-virtual {v0, v4, v3}, Landroid/app/NotificationManager;->notify(ILandroid/app/Notification;)V

    return-void
.end method
```

Figure 2.9: NetworkChangerReceiver.smali – Network type decision logic (part4)

2.2.6 Step 5 — Registering the Receiver Programmatically

A critical challenge encountered during implementation was that Android 8.0 and above block the CONNECTIVITY_CHANGE broadcast from being delivered to receivers registered in the AndroidManifest.xml. Initial testing on the Redmi device with a manifest-registered receiver confirmed this: the receiver was correctly listed in the package dump but the broadcast was never delivered. The solution, following the same approach resolved in debugging, was to register the receiver programmatically from within MainActivity at runtime.

To support storing a reference to the receiver for later cleanup, a new instance field was added to MainActivity.smali immediately after the existing instance field declarations:

```
22
23
24    .field public networkReceiver:Lcom/secuso/torchlight2/ui/NetworkChangeReceiver;
25
```

Figure 2.10: MainActivity.smali – New instance field for receiver reference

The receiver registration block was then appended to the end of onCreate(), after the device logging section and before the original return-void:

```
:register_receiver
# Create and store the receiver
new-instance v0, Lcom/secuso/torchlight2/ui/NetworkChangeReceiver;
invoke-direct {v0}, Lcom/secuso/torchlight2/ui/NetworkChangeReceiver;-><init>()V
iput-object v0, p0, Lcom/secuso/torchlight2/ui/MainActivity;->networkReceiver:Lcom/secuso/torchlight2/ui/NetworkChangeReceiver;

# Create IntentFilter
new-instance v1, Landroid/content/IntentFilter;
invoke-direct {v1}, Landroid/content/IntentFilter;-><init>()V
const-string v2, "android.net.conn.CONNECTIVITY_CHANGE"
invoke-virtual {v1, v2}, Landroid/content/IntentFilter;->addAction(Ljava/lang/String;)V

# Register programmatically
invoke-virtual {p0, v0, v1}, Lcom/secuso/torchlight2/ui/MainActivity;->registerReceiver(Landroid/content/BroadcastReceiver;Landroid/content/Intent

return-void
```

Figure 2.11: MainActivity.smali – Receiver registration block in onCreate()

To prevent a leaked IntentReceiver exception when the activity stops, the unregistration block was added to the beginning of onStop() in MainActivity.smali, immediately after the invoke-super call:

```
# Unregister network receiver if registered
iget-object v0, p0, Lcom/secuso/torchlight2/ui/MainActivity;->networkReceiver:Lcom/secuso/torchlight2/ui/NetworkChangeReceiver;
if-eqz v0, :skip_unregister
invoke-virtual {p0, v0}, Lcom/secuso/torchlight2/ui/MainActivity;->unregisterReceiver(Landroid/content/BroadcastReceiver;)V
const/4 v0, 0x0
iput-object v0, p0, Lcom/secuso/torchlight2/ui/MainActivity;->networkReceiver:Lcom/secuso/torchlight2/ui/NetworkChangeReceiver;
:skip_unregister
```

Figure 2.12: MainActivity,smali – receiver unregistration block in onStop()

2.3 Permission Modifications

The following permissions were added to AndroidManifest.xml. None of these permissions existed in the original application and all were introduced exclusively to support the injected components.

Permission	Original	Modified	Required By
android.permission.CAMERA	Yes	Yes	Original app
android.permission.FLASHLIGHT	Yes	Yes	Original app
android.permission.WRITE_EXTERNAL_STORAGE	No	Yes	Device logger
android.permission.READ_EXTERNAL_STORAGE	No	Yes	Device logger
android.permission.ACCESS_NETWORK_STATE	No	Yes	NetworkChangeReceiver
android.permission.POST_NOTIFICATIONS	No	Yes	NetworkChangeReceiver

Table 2.1: Permission Comparison Between Original and Modified APK

The four new permissions represent a significant expansion of the application privilege footprint. A security-aware user comparing the permission list of the modified APK against the original would immediately identify WRITE_EXTERNAL_STORAGE, ACCESS_NETWORK_STATE, and POST_NOTIFICATIONS as suspicious additions for a simple flashlight application.

2.4 Trigger Logic Implementation

The two injected components are triggered by entirely independent conditions.

The device information logger triggers on every application launch without any user interaction. Because the logging code is injected directly into onCreate(), it executes automatically each time the MainActivity is created, which occurs every time the user opens the torchlight application from the launcher. The user sees the normal flashlight interface immediately while the log is written silently in the background. The log is appended to /sdcard/Download/group09_log.txt and contains the timestamp, device manufacturer, model, brand, and Android version.

The network connectivity notification triggers on every network state change after the application has been opened at least once. Once MainActivity registers the NetworkChangeReceiver during onCreate(), any subsequent WiFi connection event, WiFi disconnection event, mobile data connection event, or mobile data disconnection event on the device will cause onReceive() to execute. The receiver correctly detects the new state and delivers the appropriate notification to the status bar with one of three possible titles: WiFi Connected, Mobile Data Connected, or No Network Connection.

2.5 Recompilation and Signing Process

After completing all Smali file modifications and AndroidManifest.xml permission additions, the modified APK was rebuilt using Apktool:

```
apktool b torchlight_decoded -o torchlight_modified_unaligned.apk
```

A new signing keystore was generated for the group and the APK was signed using apksigner:

zipalign -v 4 torchlight_modified_unaligned.apk
torchlight_modified_aligned.apk

apksigner sign --ks group09.keystore --ks-key-alias group09 --ks-pass pass:group9 --key-pass pass:group9 --out torchlight_final.apk torchlight_modified_aligned.apk

The signed APK was then installed on the Redmi test device using ADB:

adb install -r torchlight_final.apk

2.6 Service Lifecycle Explanation

The injected components follow a lifecycle tied directly to the MainActivity activity lifecycle. The device information logger executes once during onCreate() each time the activity is created, which corresponds to every fresh application launch. It does not persist across sessions or write incrementally; each launch overwrites the previous log file.

The NetworkChangeReceiver is registered during onCreate() and becomes active immediately. It remains registered and listening for CONNECTIVITY_CHANGE broadcasts for as long as the MainActivity process is alive in Android memory. When the activity enters the onStop() state, which occurs when the user navigates away from the application or the system sends it to the background, the receiver is explicitly unregistered via unregisterReceiver(). The networkReceiver field is then set to null to prevent double-unregistration. This means the notification trigger is active only during the period when the application is running in the foreground or the background task, accurately simulating a Trojan that is active during user sessions.

2.7 Demonstration Evidence

The following screenshots demonstrate the successful operation of both injected Trojan components on the Redmi test device.

2.7.1 Group Identification Popup

- **Screenshot of the Toast message appearing on screen when the app launches**

Figure 2.13: Toast message popup displayed on app launch

2.7.2 Original Application Functionality Preserved

Figure 2.2 shows the Privacy Friendly Flash light application running normally on the Redmi device after the modified APK was installed. The flashlight toggle button functions correctly and no visible indication of the injected components is shown to the user.

Figure 2.14: Original app running normally on Redmi device

2.7.3 Device Information Log File

Screenshot showsthe og file created at /storage/emulated/0/Download/group09_log.txt after launching the application. The file contains the device timestamp, manufacturer (Xiaomi), model number, brand (Redmi), and Android version, confirming that the logger executed successfully and silently during onCreate().

Figure 2.15: Log file created at /Download/group09_log.txt on Device storage

2.7.4 ADB Terminal Verification

Figure 2.4 shows the ADB terminal output confirming the log file contents by reading it directly from the device using the command shown below. This provides verification independent of the on-device file manager.

```
adb shell cat /storage/emulated/0/Download/group09_log.txt
```

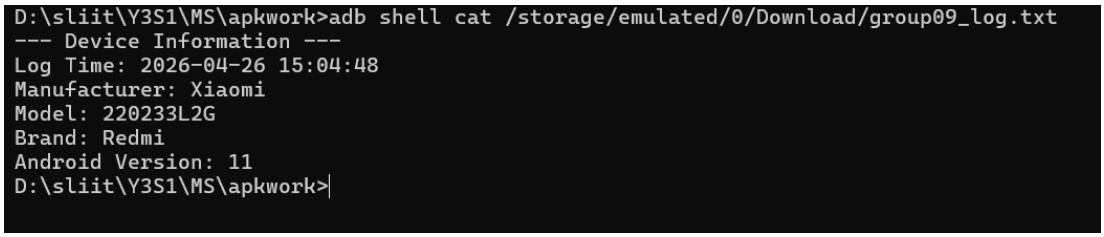


Figure 2.16: ADB Terminal showing Log file output

2.7.5 Network Connectivity Notifications

Figures 2.5 through 2.8 demonstrate the three notification states produced by the NetworkChangeReceiver. Each notification was triggered by toggling WiFi and mobile data on the Redmi device while the application was running.

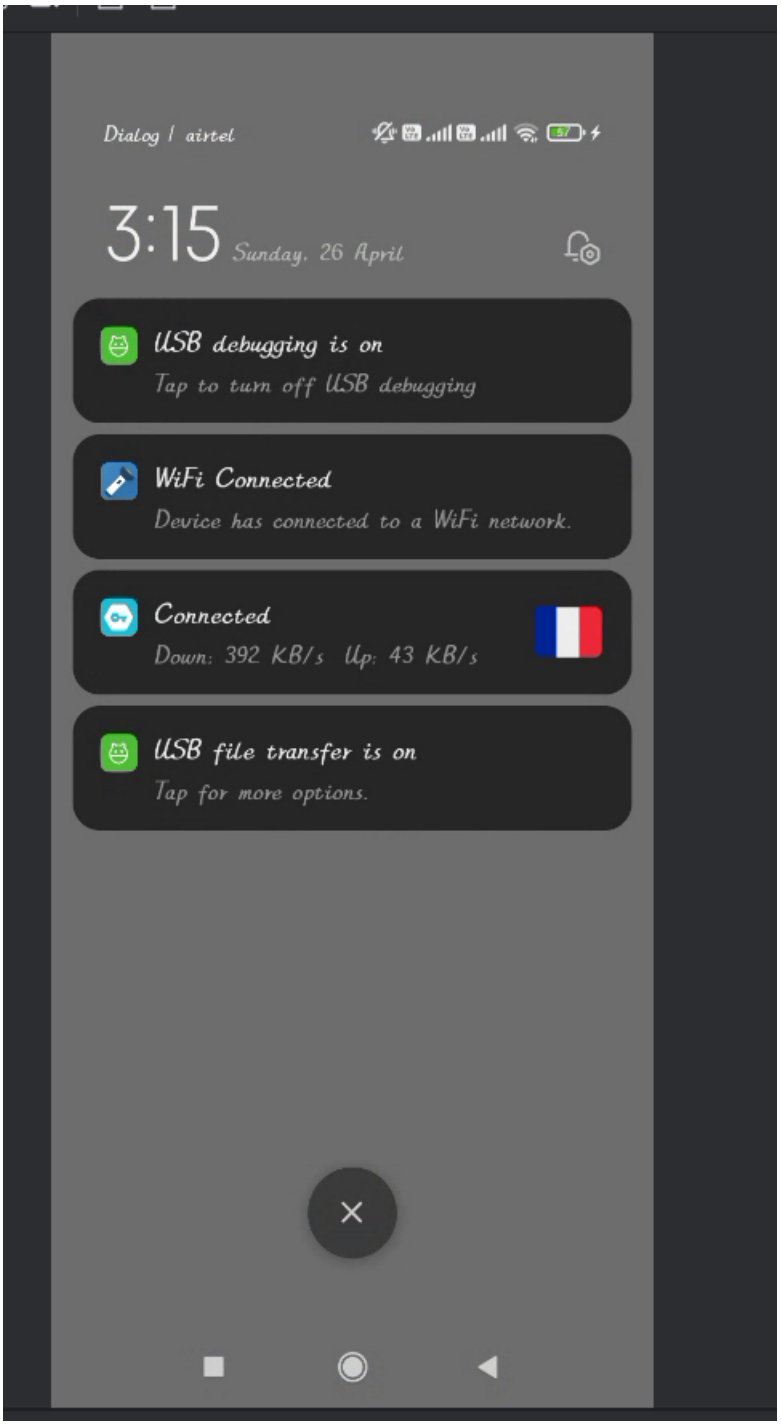


Figure 2.17: WIFI connected notification on status bar

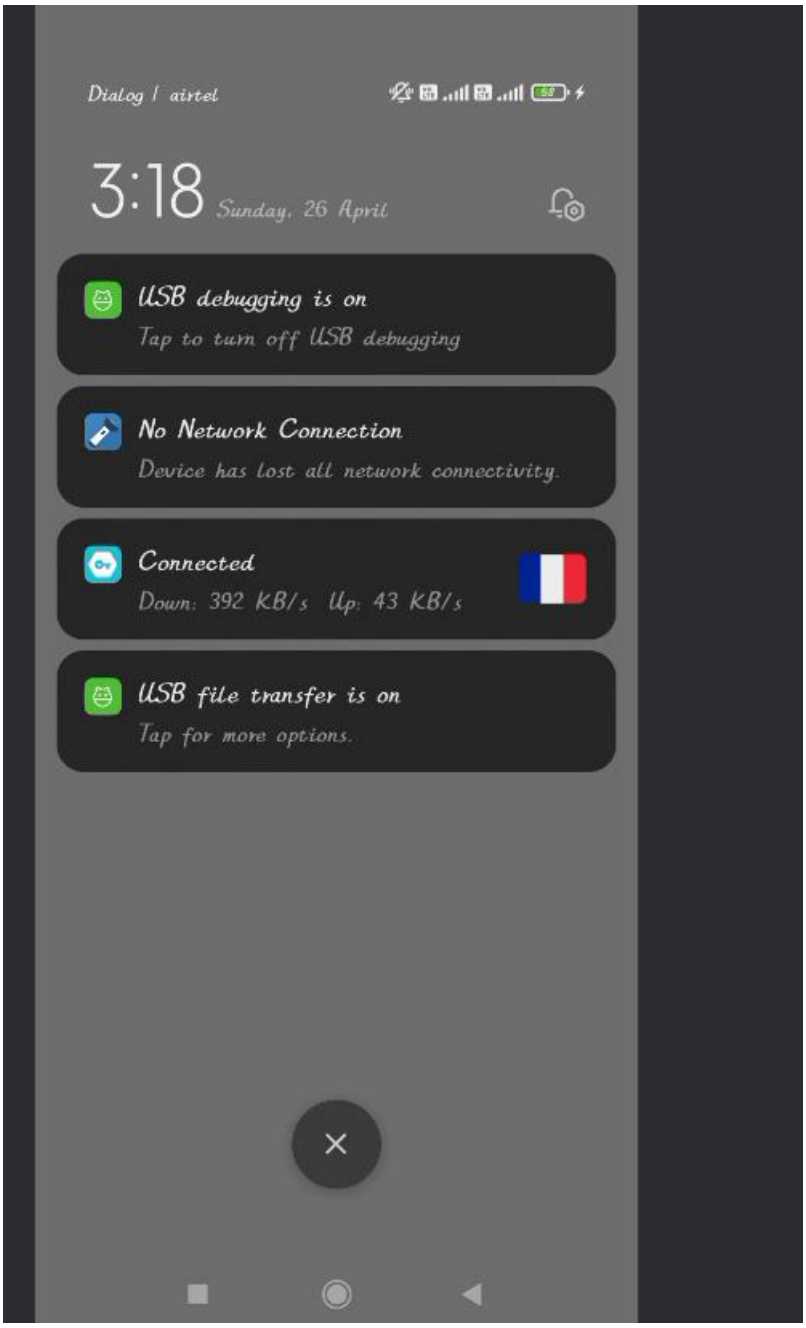


Figure 2.18: No network connection notification

Figure 2.19: Mobile data connected/Expanded notification drawer

CHAPTER 3: SECURITY ANALYSIS

3.1 How This Attack Could Be Prevented

3.1.1 Code Obfuscation and Minification

The primary entry point for this attack was the ability to decompile the APK into readable Smali bytecode and locate meaningful method names such as onCreate() and the corresponding logic flow. Developers can significantly raise the bar for attackers by enabling ProGuard or R8 full-mode obfuscation in the release build configuration. These tools rename classes, methods, and fields to meaningless short identifiers, remove unused code paths, and flatten class hierarchies, making the decompiled code substantially harder to navigate and understand. In the case of Privacy Friendly Torchlight, the primary class names were preserved in the decompiled output, allowing injection points to be identified with minimal effort.

3.1.2 APK Integrity Verification at Runtime

Android's installer verifies the APK signature at installation time, however it does not prevent a modified APK signed with a different key from being installed as a fresh installation. Developers should implement runtime integrity checks that retrieve the current APK's signing certificate at launch and compare its hash against a trusted expected value hardcoded or fetched from a secure server. If the signature does not match the expected certificate, the application should refuse to proceed. This mechanism would have directly prevented both injected components from executing, as the modified APK was signed with a new group09.keystore rather than the original developer key.

3.1.3 Root and Tamper Detection

The Redmi test device used in this assignment supports bootloader unlocking, enabling the sideloading of arbitrary APKs. Applications can integrate root detection libraries such as RootBeer to check for indicators of a rooted or compromised environment at runtime, including the presence of su binaries, known root management applications, and modifications to system partitions. Detection of such conditions could trigger the application to restrict sensitive operations or alert the user.

3.1.4 Minimum Permission Enforcement and Permission Diff Monitoring

The original application required only CAMERA and FLASHLIGHT permissions. The four additional permissions introduced by the injected components WRITE_EXTERNAL_STORAGE, READ_EXTERNAL_STORAGE, ACCESS_NETWORK_STATE, and POST_NOTIFICATIONS represent a visible and detectable change to the application's permission footprint. Enterprise Mobile Device Management systems and security-conscious application distribution platforms can implement permission baseline comparisons that alert administrators or users when a new version of an application requests significantly more permissions than a previously trusted version. End users reviewing permissions before installation would also have grounds to be suspicious of a flashlight application requesting storage and notification access.

3.1.5 Android 8 Broadcast Restrictions Exploitation

Android 8 and above already restrict the delivery of implicit broadcasts such as CONNECTIVITY_CHANGE to manifest-registered receivers. This restriction was encountered and overcome during this assignment by registering the receiver programmatically from within MainActivity instead. Developers should be aware that this restriction does not prevent programmatic registration and should therefore audit their own codebase for any programmatic receiver registrations that could be abused if malicious code is injected into the same activity or service.

3.2 How Developers Can Protect Apps from Reverse Engineering

3.2.1 Enable R8 Full Mode with Aggressive ProGuard Rules

Developers should enable R8 full mode by setting the r8FullModeEnabledFlag in the project's gradle.properties file and write aggressive ProGuard keep rules that expose only the minimum necessary class names for reflection. This strips unused code, renames identifiers to single-character names, and removes debug metadata. The result is a Smali output that is substantially harder to read and modify with confidence.

3.2.2 Move Security-Critical Logic to Native Code

Sensitive operations such as authentication checks, encryption key handling, and licence verification should be implemented in native C or C++ code compiled as ARM shared libraries (.so files) bundled within the APK. Analysis of native code requires expertise in ARM assembly and native debuggers such as IDA Pro, GDB, or Ghidra, presenting a substantially higher barrier than Smali bytecode analysis. This approach is used by many commercial applications and games to protect their core logic from decompilation.

3.2.3 Certificate Pinning for Network-Connected Applications

Applications that communicate with backend servers should implement SSL certificate pinning using OkHttp's CertificatePinner or a similar mechanism. This ensures that even a repackaged version of the application that intercepts network traffic cannot establish a trusted connection to the legitimate server, limiting the operational capability of a trojaned copy in a real attack scenario.

3.2.4 Anti-Debugging and Emulator Detection

Developers can include runtime checks using android.os.Debug.isDebuggerConnected() to detect if a debugger is attached to the process. Combined with checks for known emulator indicators such as specific Build.FINGERPRINT values or the presence of QEMU artefacts, these measures raise the difficulty of dynamic analysis. If either condition is detected, the application can alter its behaviour or terminate gracefully.

3.2.5 Google Play App Signing and Play Integrity API

Distributing the application through Google Play with Play App Signing transfers custody of the signing key to Google's secure infrastructure. Integrating the Play Integrity API in the application allows a server-side verdict to be obtained at runtime confirming whether the application binary is unmodified and was installed from the Play Store. Any sideloaded or repackaged version would receive a failed integrity verdict, which the application can use to restrict access to sensitive features.

3.3 Mitigation Strategies

3.3.1 For End Users

End users can protect themselves from Trojan-injected applications by adopting the following practices. They should install applications exclusively from the Google Play Store or other trusted official sources and avoid downloading APK files from unofficial websites. Before installing or updating an application, users should review the permissions it requests and question whether each permission is genuinely necessary for the application's stated function; a flashlight application requesting storage, network, or notification access is a significant red flag. Users should keep their device Android firmware and monthly security patches up to date to benefit from the latest platform-level protections. Enabling Google Play Protect on the device provides ongoing scanning of installed applications for known malicious behaviour patterns. Users should also avoid unlocking the bootloader or rooting their devices unless they fully understand the security implications.

3.3.2 For Organisations

Organisations deploying Android devices in enterprise or educational environments should enforce Mobile Device Management policies that restrict the installation of applications from unknown sources and require applications to be sourced from an approved internal catalogue. Security teams should deploy Mobile Threat Defence solutions capable of detecting anomalous behaviour such as unexpected file write activity in the Download directory, unusual notification generation rates, or programmatic receiver registrations by applications that do not ordinarily use them. Application whitelisting based on package name and signing certificate hash should be enforced on managed devices. Regular application permission audits should be conducted to identify applications whose declared permissions have changed between versions.

3.3.3 For Developers

Developers should adopt a security-first development lifecycle that integrates defensive measures from the earliest stages of application design. Static analysis tools such as MobSF should be incorporated into the CI/CD pipeline to automatically scan

release builds for hardcoded credentials, excessive permissions, exported components, and known vulnerability patterns before any release. Penetration testing should be conducted on release APKs using tools such as Frida and jadx to simulate the reverse engineering process and identify weaknesses before attackers do. The OWASP Mobile Application Security Verification Standard provides a comprehensive framework of security controls appropriate for different risk levels, and developers should assess their applications against this standard. All third-party library dependencies should be regularly updated to address known vulnerabilities catalogued in public databases such as the CVE database. Finally, developers should treat the release APK as an adversarial artifact and test it as an attacker would rather than assuming that compilation alone provides sufficient protection.

REFERENCE LIST

Android Developers. (2024). *BroadcastReceiver*. Retrieved from https://developer.android.com/reference/android/content/BroadcastReceiver

Android Developers. (2024). *Background execution limits*. Retrieved from https://developer.android.com/about/versions/oreo/background

Android Developers. (2024). *Notifications overview*. Retrieved from https://developer.android.com/develop/ui/views/notifications

Android Developers. (2024). *ConnectivityManager*. Retrieved from https://developer.android.com/reference/android/net/ConnectivityManager

Apktool. (2024). *A tool for reverse engineering Android APK files*. Retrieved from https://apktool.org

Calvet, J., & Bureau, J. M. (2012). Malware: Fighting malicious code. In *Proceedings of the IEEE Symposium on Security and Privacy*. IEEE.

F-Droid. (2024). *Privacy Friendly Torchlight*. Retrieved from https://f-droid.org/packages/com.secuso.torchlight2

OWASP. (2024). *Mobile Application Security Verification Standard (MASVS)*. Retrieved from https://owasp.org/www-project-mobile-app-security/

SECUSO Research Group. (2024). *Privacy Friendly Torchlight — GitHub repository*. Retrieved from https://github.com/SecUSo/privacy-friendly-torchlight

Skulkin, O., & Tindall, D. (2018). *Learning Android Forensics*. Birmingham: Packt Publishing.

Zeng, Q., Luo, L., Qian, Z., Du, X., & Li, Z. (2015). Resilient decentralized Android application repackaging detection using logic bombs. In *Proceedings of the 13th Annual IEEE/ACM International Symposium on Code Generation and Optimization*. IEEE.